

Proof Systems in Practice

Simons Institute Boot Camp, May 2025

Justin Thaler
Georgetown University and a16z crypto research

Important Disclosures

The views expressed here are those of the individual AH Capital Management, L.L.C. (“a16z”) personnel quoted and are not the views of a16z or its affiliates. Certain information contained in here has been obtained from third-party sources, including from portfolio companies of funds managed by a16z. While taken from sources believed to be reliable, a16z has not independently verified such information and makes no representations about the enduring accuracy of the information or its appropriateness for a given situation.

This content is provided for informational purposes only, and should not be relied upon as legal, business, investment, or tax advice. You should consult your own advisers as to those matters. References to any securities, digital assets, tokens, and/or cryptocurrencies are for illustrative purposes only and do not constitute a recommendation to invest in any such instrument nor do such references constitute an offer to provide investment advisory services. Furthermore, this content is not directed at nor intended for use by any investors or prospective investors, and may not under any circumstances be relied upon when making a decision to invest in any fund managed by a16z. (An offering to invest in an a16z fund will be made only by the private placement memorandum, subscription agreement, and other relevant documentation of any such fund and should be read in their entirety.) Any investments or portfolio companies mentioned, referred to, or described are not representative of all investments in vehicles managed by a16z, and there can be no assurance that the investments will be profitable or that other investments made in the future will have similar characteristics or results. A list of investments made by funds managed by Andreessen Horowitz (excluding investments for which the issuer has not provided permission for a16z to disclose publicly as well as unannounced investments in publicly traded digital assets) is available at <https://a16z.com/investments/>.

Charts and graphs provided within are for informational purposes solely and should not be relied upon when making any investment decision. Past performance is not indicative of future results. The content speaks only as of the date indicated. Any projections, estimates, forecasts, targets, prospects, and/or opinions expressed in these materials are subject to change without notice and may differ or be contrary to opinions expressed by others. Please see <https://a16z.com/disclosures> for additional important information.

What is a SNARK?

- Prover **P** claims to know witness w satisfying some property.
 - For example: “I know an w such that $\text{SHA256}(w) = 0$ ”
- Trivial proof: **P** sends w to **V**, who directly checks it satisfies the claimed property.

What is a SNARK?

- Prover **P** claims to know witness w satisfying some property.
 - For example: “I know an w such that $\text{SHA256}(w) = 0$ ”
- Trivial proof: **P** sends w to **V**, who directly checks it satisfies the claimed property.
- A SNARK (**S**uccinct **N**on-interactive **A**rgument of **K**nowledge) achieves the same, but with better costs to **V**.
 - SNARK proof π **must** be shorter than the witness w .
 - This is what “succinct” means.
 - Ideally, checking π is faster than checking w directly.
 - Called “work-saving” for **V**.

SNARK tooling pipelines

General-Purpose SNARKs: Standard Paradigm

- Start with a computer program ψ written in high-level programming language (C, Rust, etc.).
 - Hash pre-image example:
 - Program takes w as input, applies SHA2 to it, and checks that output equals y .
- Step 1: Turn ψ into an equivalent model amenable to probabilistic checking.
 - Typically some variant of circuit-satisfiability.
 - The “circuit” is called an **intermediate representation (IR)**.
 - Really, it is a “constraint system”.
 - Called the **Front End** of the system.
- Step 2: Run a SNARK for circuit-satisfiability.
 - Called the **Back End** of the system.

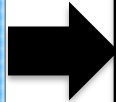
```

blink.c
/*****
 * Author: Leon Bonatti
 * Filename: blink.c
 * Date: 11/10/21
 */

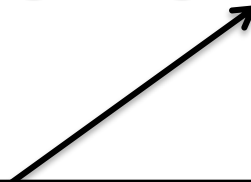
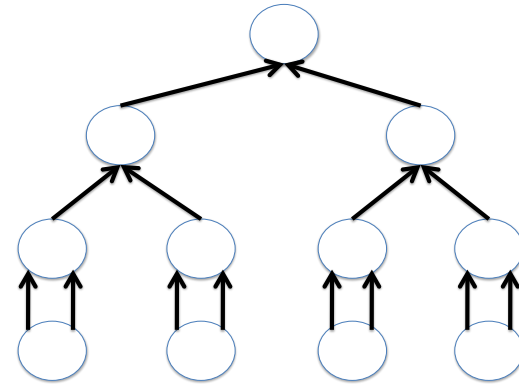
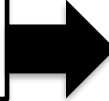
#define F_CPU 1000000
#include <avr/io.h>
#include <util/delay.h>
#include <../lib/avr/io.h>

int main(void)
{
    DDRD |= (1<<5);
    PORTD |= (1<<5);
    while(1)
    {
        if (bit_is_low(PIND))
        {
            PORTD |= (1<<5);
        }
        else
        {
            PORTD &= ~(1<<5);
        }
    }
    return 0;
}

```



Front
End



P and V run SNARK (back end) for circuit-satisfiability.

SNARK backend design details

Key Paradigm for SNARK Design

1. Give a “**polynomial IOP**” (**PIOP**) for circuit-satisfiability.
2. Combine with **polynomial commitment scheme** (**PCS**) to get succinct interactive argument.
3. Apply **Fiat-Shamir transformation** to render non-interactive.

Key Paradigm for SNARK Design

1. Give a “**polynomial IOP**” (**PIOP**) for circuit-satisfiability.
 2. Combine with **polynomial commitment scheme** (**PCS**) to get succinct interactive argument.
 3. Apply **Fiat-Shamir transformation** to render non-interactive.
- This is how all SNARKs are designed other than “linear-PCP based” ones (GGPR13, Pinocchio, Groth16, etc.)
 - Even linear-PCP based SNARKs use techniques **very** similar to certain polynomial commitments (KZG).

What is a Polynomial IOP?

1. **P**'s first message in the protocol is a **polynomial** h .
 - **V** does **not** learn h in full.
 - The description size of h is as large as the circuit.
 - Rather, **V** is permitted to evaluate h at, say, **one** point.
2. After that, **P** and **V** execute a standard interactive proof.

What is a Polynomial IOP?

1. **P**'s first message in the protocol is a **polynomial** h .
 - **V** does **not** learn h in full.
 - The description size of h is as large as the circuit.
 - Rather, **V** is permitted to evaluate h at, say, **one** point.
2. After that, **P** and **V** execute a standard interactive proof.
 - Often, h "encodes" the witness that **P** claims to know, and Step 2 is meant to check that h is indeed a valid witness.
 - From **V**'s perspective, Step 2 is a reduction, from the task of checking some **global** claim about h (involving **many** evaluations of h), to a **local** claim (a single evaluation of h).

Two classes of polynomial IOPs

1. Based on univariate polynomials and “quotienting”.
 - Constant-round.
2. Based on multilinear polynomials and the sum-check protocol.
 - Logarithmically many rounds.

Two classes of polynomial IOPs

1. Based on univariate polynomials and “quotienting”.
 - Constant-round.
2. Based on multilinear polynomials and the sum-check protocol.
 - Logarithmically many rounds.
- High-level comparison: (1) is better for **V** costs, (2) is better for **P** costs.
 - The proof sizes of (2) are still just logarithmic (or maybe polylog).
 - Concretely, a few KBs up to dozens of KBs.
 - Can be reduced further via SNARK composition.

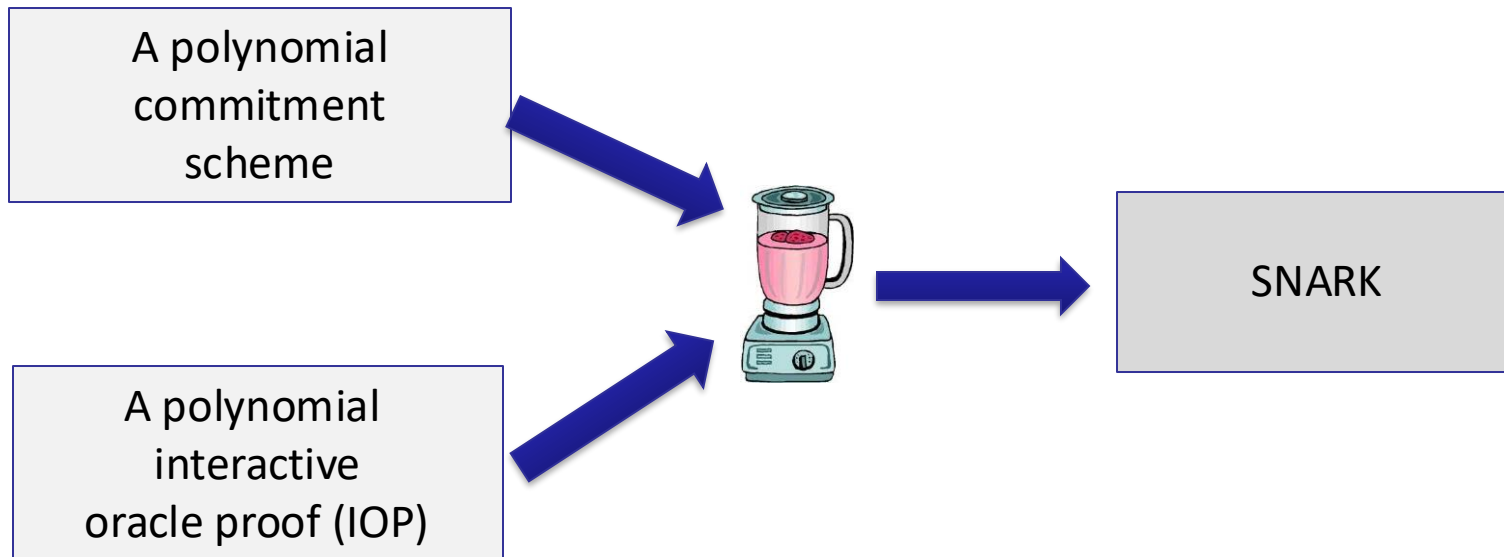
What is a Polynomial Commitment Scheme?

- High-level idea:
 - **P** binds itself to a polynomial h by sending a short string $\text{Com}(h)$.
 - **V** can choose x and ask **P** to evaluate $h(x)$.
 - **P** sends y , the purported evaluation, plus a proof π that y is consistent with $\text{Com}(h)$ and x .
- Goals:
 - **P** cannot produce a convincing proof for an incorrect evaluation.
 - $\text{Com}(h)$ and π are short and easy to generate; π is easy to check.

Polynomial Commitment Schemes: Four Classes

1. Based on pairings + trusted setup (**not** transparent **nor** post-quantum).
 - e.g., **KZG10**, PST13, ZGKPP18, HyperKZG, Zeromorph.
 - Unique property: very short proofs and fast verifier.
2. Based on discrete logarithm (transparent, **not** post-quantum).
 - e.g., **IPA/Bulletproofs**, Hyrax, Dory.
3. Based on hashing (transparent **and** post-quantum)
 - e.g., **FRI**, Ligerio, Brakedown, FRI-Binius, BaseFold, Blaze, STIR, WHIR
4. Lattices (transparent **and** post-quantum)
 - LaBRADOR, Greyhound, etc. Still maturing.

Let's build an efficient SNARK



SNARK performance today

SNARK performance

- SNARK **V** is generally cheap.
 - Much less work than trivial proof system (directly checking **w** for validity).
 - But considerable variation in costs.
 - Groth16 or Plonk proofs (using KZG commitments) are ~200k gas to verify on Ethereum.
 - Under 1 KB.
 - FRI proofs are many millions of gas.
 - ~100 KBs

SNARK performance

- SNARK **P** is generally expensive.
 - **P** does ~6 orders of magnitude more work than directly checking **w** for validity.
 - This overhead can be parallelized, but million-fold slowdowns are still limiting.
 - Example: proving validity of transactions in an Ethereum block today in ~12s takes many dozens of GPUs.
 - Ethereum wants to 100x the amount of computation in a block over the next 4 years.

SNARK performance

- SNARK **P** is generally expensive.
 - **P** does ~6 orders of magnitude more work than directly checking **w** for validity.
 - This overhead can be parallelized, but million-fold slowdowns are still limiting.
 - Example: proving validity of transactions in an Ethereum block today in ~12s takes many dozens of GPUs.
 - Ethereum wants to 100x the amount of computation in a block over the next 4 years.
 - Overhead now falling to ~5 orders of magnitude.
 - Thanks to techniques I will cover today.
 - Prediction: within one-to-two years, it will be ~4 orders of magnitude.

Focusing on P time

- I will generally focus on P time.
- Why?
 - It's the main limiting (performance) factor for SNARK applicability.
 - Recursion can give "the best of both worlds":
 - Run a **fast-prover / large-proof** SNARK $\rightarrow$ obtain proof π .
 - Turn that SNARK's verifier into a circuit C .
 - Feed C to a **tiny-proof** SNARK $\rightarrow$ tiny proof of "I know π ."
 - C is smaller than the original statement (assuming the fast-prover SNARK has a sublinear-time verifier), so the slow prover tackles only a small circuit.
 - So overall proving stays quick.

Differences between theory and practice

- Cryptographic assumptions
 - Practitioners are fine with RO, and standard assumptions like discrete log.
 - Many go further:
 - Algebraic Group Model and other idealized models
 - Heuristic security, e.g., recursive SNARKs not provably secure even in RO
 - Some simply *assume* current attacks are optimal $\rightarrow$ saves $\approx 2\times$ proof size
- Logarithmic factors and even constants matter (a lot)
 - We've long had SNARKs near-optimal up to log factors (verifier & prover)
 - ...but plenty of room for further gains
- Simplicity matters (a lot)



I WANT YOU
TO ADVANCE
SNARK DESIGN

Warning: controversial opinion incoming

Cost analysis

- In most of TCS, performance analysis $\neq$ implementation reality. You have to run the algorithm on real inputs to see how it performs.
- But SNARKs are a rare exception.
- **We can *exactly* count prover work:** number of field ops, group ops, etc.
- These counts **translate directly to CPU cycles**. Not just asymptotically, *concretely*.
- Why this holds:
 - Prover (for many SNARKs) is nearly data-oblivious.
 - Dominant operations have predictable cost.
- Consequence: SNARK performance is governed by theoretical performance models that are *predictively accurate*.
- TL;DR: You don't need to run the prover to know if it's efficient, you just need to count.
 - Takes some getting used to, but it's still theory like we know and love.

Outline for the rest of the talk

Remainder of the Talk

- Part 1: Foundational techniques (2020 and prior)
 - Polynomial commitments from each class:
 - Hyrax as a detailed case study
 - Representative PIOPs from each class:
 - Univariate polynomials and quotienting
 - Sum-check and the “zero-check PIOP”
- Part 2: Newer trends and developments:
 - zkVMs
 - Lookup arguments aka batch-evaluation arguments
 - Streaming provers to keep memory usage in check (instead of using SNARK recursion)
 - Open directions

Background on Cryptographic Groups

Multi-exponentiations

- In a multiplicative group G , a **multi-exponentiation** of size n computes an expression of the form

$$\prod_{i=1}^n g_i^{x_i}$$

- Here, each $g_i \in G$, and each x_i is a field element (in the scalar field of G , which is typically of size about 2^{256}).
- Also called an **MSM (multi-scalar multiplication)** based on additive group notation.
 - $\prod_{i=1}^n g_i^{x_i}$ becomes $\sum_{i=1}^n g_i^{x_i} := \langle x, g \rangle$.
 - Can think of the MSM as the “inner product” of a vector x of **field elements** with a vector g of **group elements**.

Pedersen commitments

- Commitment key: n **random** group elements $(g_1, \dots, g_n)$.
 - In practice, each g_i is generated via a PRG evaluation (plus running "hash-to-curve").
- The Pedersen commitment to a vector $\mathbf{x} = (x_1, \dots, x_n)$ of field elements is:

$$\prod_{i=1}^n g_i^{x_i}$$

- Or in additive group notation: $\langle \mathbf{x}, \mathbf{g} \rangle$

Multi-exponentiation algorithms

- Naïve algorithm does n group exponentiations and n group multiplications.
 - **Each exponentiation can be ~400x slower than a group multiplication.**
- Pippenger's algorithm saves a factor of about $\log(n)$.
 - Can be over 10x in practice.

Multi-exponentiation algorithms

- Naïve algorithm does n group exponentiations and n group multiplications.
 - **Each exponentiation can be ~400x slower than a group multiplication.**
- Pippenger's algorithm saves a factor of about $\log(n)$.
 - Can be over 10x in practice.
- If all exponents are “small”, can save **another** factor of 10x.
 - Analogous to how computing $g_i^{2^{16}}$ is 10x faster than computing $g_i^{2^{160}}$.
 - 16 squarings vs. 160.

Multi-exponentiation algorithms in 3 bullets

- Summary of time to compute $\prod_{i=1}^n g_i^{x_i}$:
 - Any $x_i = 0$ is “free”: does not affect the result at all.
 - Any $x_i = 1$ takes exactly one group multiplication to process.
 - Non-obvious: any “small” x_i (in roughly $\{0, 1, \dots, n\}$) also takes about one group multiplication to process.

Multi-exponentiation algorithms in 3 bullets

- Summary of time to compute $\prod_{i=1}^n g_i^{x_i}$:
 - Any $x_i = 0$ is “free”: does not affect the result at all.
 - Any $x_i = 1$ takes exactly one group multiplication to process.
 - Non-obvious: any “small” x_i (in roughly $\{0, 1, \dots, n\}$) also takes about one group multiplication to process.
- Call this cost profile “**pay-per-bit commitment costs**”.

Multi-exponentiation algorithms in 3 bullets

- Summary of time to compute $\prod_{i=1}^n g_i^{x_i}$:
 - Any $x_i = 0$ is “free”: does not affect the result at all.
 - Any $x_i = 1$ takes exactly one group multiplication to process.
 - Non-obvious: any “small” x_i (in roughly $\{0, 1, \dots, n\}$) also takes about one group multiplication to process.
- Warning: many libraries are not yet optimized for committing to small values

Bit-width of committed values	halo2curves commit (ms)	Nova commit (ms)
1	226.72	8.66
10	249.75	15.73
16	370.24	34.7 <u>6</u>
32	417.82	52.37
64	282.60	98.53

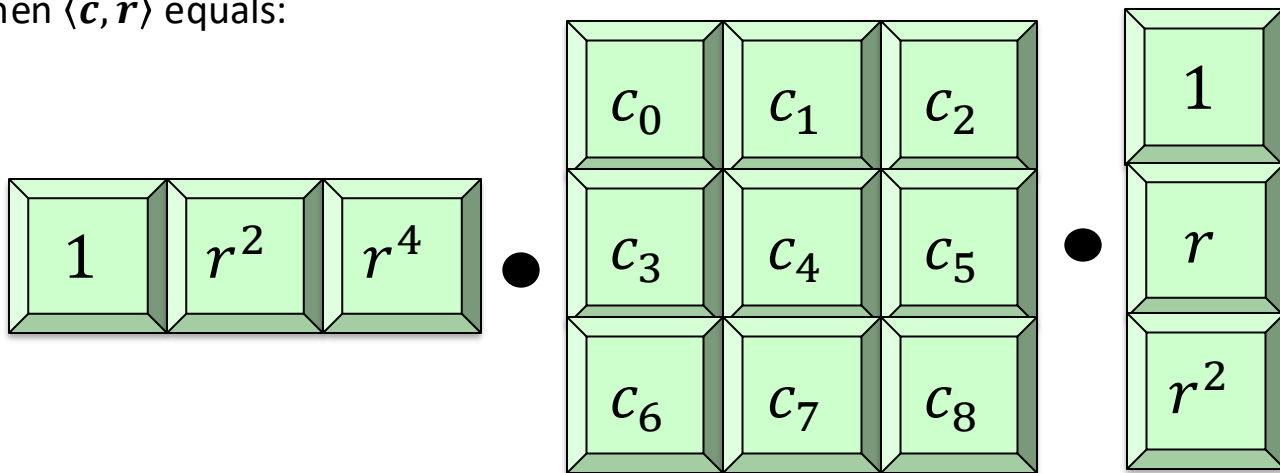
Hyrax Commitments

Hyrax costs

- High **V** costs:
 - Commitment size: $\sqrt{n}$ group elements (100 KBs up to ~1 MB)
 - Proof size: $\sqrt{n}$ field elements
 - **V** work: 2 MSMs of size $\sqrt{n}$
- Low **P** costs:
 - Commitment key size: $\sqrt{n}$ group elements.
 - In contrast, many other curve-based commitment schemes have commitment keys of size n .
 - Transparent. Commitment key is random group elements, which can be generated locally by **P**.
 - Prover time:
 1. One MSM of size n involving coefficients of p (to commit).
 2. Opening proof just involves evaluating p at the requested point r .
 - Applies directly to multilinear and univariate polynomials, not just univariate.

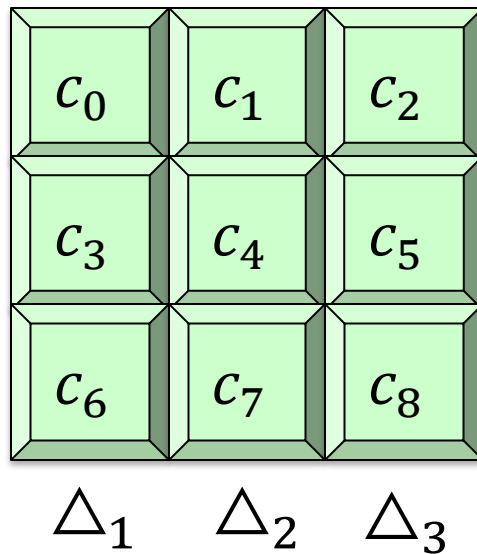
Hyrax workings

- Idea: exploit “vector-matrix-vector multiplication structure” in polynomial evaluation queries
 - Let $p(X) = \sum_{i=0}^n c_i X^i$.
 - $p(r) = \langle \mathbf{c}, \mathbf{r} \rangle$ where $\mathbf{c} = (c_0, \dots, c_n)$ and $\mathbf{r} = (r^0, r^1, \dots, r_n)$.
 - $\mathbf{r}$ has **multiplicative structure**.
- Let $n = 8$. Then $\langle \mathbf{c}, \mathbf{r} \rangle$ equals:



Hyrax commitment

- Commitment key: $\mathbf{g} = (g_1, g_2, \dots, g_{\sqrt{n}})$, vector of random group elements
- Let $p(X) = \sum_{i=0}^n c_i X^i$.
 - View $\mathbf{c}$ as a $\sqrt{n} \times \sqrt{n}$ matrix.
 - The commitment to p comprises the Pedersen commitment to each column (using the same key $\mathbf{g}$ for each).



Hyrax evaluation proofs

- **Let's use additive group notation.**
- Pedersen commitments are homomorphic:
 - Given two commitments Δ_1, Δ_2 to vectors $\mathbf{u}_1, \mathbf{u}_2$ and two field elements a_1, a_2 ,
 - $a_1 \Delta_1 + a_2 \Delta_2$ is a commitment to $a_1 \cdot \mathbf{u}_1 + a_2 \cdot \mathbf{u}_2$.

Hyrax evaluation proofs

- Let M be the “coefficient matrix” of p .
- Recall $p(r) = \mathbf{b}^T \cdot M \cdot \mathbf{a}$ for two vectors $\mathbf{a}, \mathbf{b}$ in $\mathbf{F}^{\sqrt{n}}$.

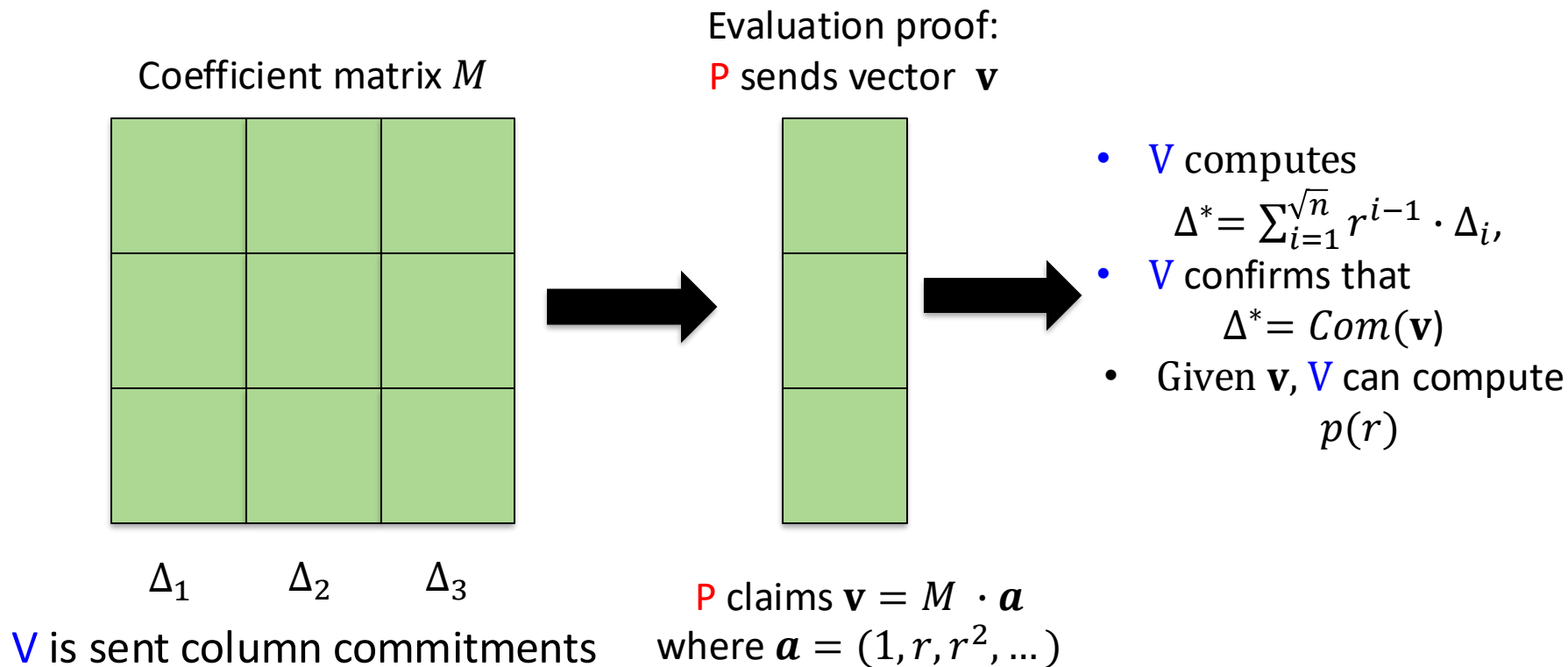
Hyrax evaluation proofs

- Let M be the “coefficient matrix” of p .
- Recall $p(r) = \mathbf{b}^T \cdot M \cdot \mathbf{a}$ for two vectors $\mathbf{a}, \mathbf{b}$ in $\mathbf{F}^{\sqrt{n}}$.
- The evaluation proof is just a vector $\mathbf{v} \in \mathbf{F}^{\sqrt{n}}$ claimed to equal $M \cdot \mathbf{a}$.
- **P** must compute $M \cdot \mathbf{a}$ anyway, “en route” to evaluating $p(r)$.

Hyrax evaluation proofs

- Let M be the “coefficient matrix” of p .
- Recall $p(r) = \mathbf{b}^T \cdot M \cdot \mathbf{a}$ for two vectors $\mathbf{a}, \mathbf{b}$ in $\mathbf{F}^{\sqrt{n}}$.
- The evaluation proof is just a vector $\mathbf{v} \in \mathbf{F}^{\sqrt{n}}$ claimed to equal $M \cdot \mathbf{a}$.
- **P** must compute $M \cdot \mathbf{a}$ anyway, “en route” to evaluating $p(r)$.
- **Checking that $\mathbf{v}$ indeed equals $M \cdot \mathbf{a}$:**
 - **V** uses homomorphism to compute a commitment Δ^* to $M \cdot \mathbf{a}$ from the commitments to the columns of M .
 - i.e., if $\Delta = (\Delta_1, \dots, \Delta_{\sqrt{n}})$ is the vector of row commitments, then $\langle \Delta, \mathbf{a} \rangle$ is a commitment to $M \cdot \mathbf{a}$.
 - **V** confirms that $\mathbf{v}$ opens the commitment Δ^* .
 - i.e., that $\langle \mathbf{v}, \mathbf{g} \rangle = \Delta^*$.
- **V** outputs $p(r) = \langle \mathbf{v}, \mathbf{a} \rangle$.

Hyrax evaluation proofs



Addressing Hyrax's downsides

- Recall the BAD aspects of Hyrax
 - Commitment size: $\sqrt{n}$ group elements (100 KBs up to ~1 MB)
 - Proof size: $\sqrt{n}$ field elements
 - $\mathbf{V}$ work: 2 MSMs of size $\sqrt{n}$
- Bulletproofs/IPA:
 - GOOD:
 - Commitment size is just one group element, and proof size to $2 \log n$ group elements (~2 KBs).
 - Transparent and homomorphic (like Hyrax).
 - BAD:
 - Size n commitment key.
 - $\mathbf{V}$ time and $\mathbf{P}$ time is n scalar multiplications (**very slow**).

Addressing Hyrax's downsides

- Recall the BAD aspects of Hyrax
 - Commitment size: $\sqrt{n}$ group elements (100 KBs up to ~1 MB)
 - Proof size: $\sqrt{n}$ field elements
 - $\mathcal{V}$ work: 2 MSMs of size $\sqrt{n}$
- Dory: roughly achieves the **best** of Hyrax and Bulletproofs:
 - Commitment size: one group element
 - Commitment key: $2\sqrt{n}$ random group elements
 - Proof size: $O(\log n)$ group elements (~15 KBs)
 - Concretely ~10x more than Bulletproofs but asymptotically the same.
 - $\mathcal{P}$ time similar to Hyrax's.
 - Extra work for $\mathcal{P}$ compared to Hyrax is computing a “multi-pairing” of size $\sqrt{n}$.
 - This is a low-order cost for $\mathcal{P}$ time as long as n is large enough.
 - Transparent and homomorphic, like both Hyrax and Bulletproofs.
 - $\mathcal{V}$ time: $O(\log n)$ scalar multiplications.
 - **Better than both** Bulletproofs and Hyrax.

Addressing Hyrax's downsides

- Recall the BAD aspects of Hyrax
 - Commitment size: $\sqrt{n}$ group elements (100 KBs up to ~1 MB)
 - Proof size: $\sqrt{n}$ field elements
 - $\mathbb{V}$ work: 2 MSMs of size $\sqrt{n}$
- Dory idea: Since the Hyrax commitment is big ($\sqrt{n}$ group elements), don't have that be the “final” commitment.
 - Instead, **commit to the Hyrax commitment**.
 - Homomorphically committing to a vector of **group elements** (instead of field elements) requires a “modified Pedersen commitment” that uses pairings.
 - For evaluation proofs, use a Bulletproofs-like protocol to have the prover prove it knows:
 - A Hyrax commitment that opens the Dory commitment.
 - A Hyrax evaluation proof attesting to the claimed value at r .

Hashing-based commitments

How Hashing-Based Commitments Work

- All hashing-based commitments work as follows.
 - **P** takes (the coefficients of) the polynomial to be committed, encodes them with some linear error-correcting code, and Merkle-hashes the result.
 - For evaluation proofs **V** requests that **P** send certain random linear combinations of the committed values.
 - And **V** “spot-checks” $O(\lambda)$ random entries of those purported linear combinations for consistency with the original commitment.

P time for hashing-based commitments

- Downside: 0s are not free to commit to.
 - For **any** non-zero vector x , encoding x with a good linear code leads to a **constant fraction** of non-zero entries.
 - And then that codeword must be hashed by **P**.
- Works focused on fast commitments to small values:
 - Binius, FRI-Binius, Blaze.
 - These work over binary fields.
 - The idea:
 - Most SNARKs must work over a 128+-bit field to ensure adequate security.
 - Avoid assigning a full 128-bit field element to each committed value.
 - Instead, “pack” many small values into a single field element before committing.

Ajtai Commitments (Lattices)

Ajtai commitments

- Recall Pedersen commitments:
 - Commitment key: n **random** group elements $(g_1, \dots, g_n)$.
 - The Pedersen commitment to a vector $\mathbf{x} = (x_1, \dots, x_n)$ of field elements (in additive group notation) is: $\langle \mathbf{x}, \mathbf{g} \rangle = \sum_{i=1}^n x_i \cdot g_i$.
- Ajtai commitments are similar, except that random group elements $(g_1, \dots, g_n)$ get replaced with vectors of random **ring** elements.
 - The ring elements are themselves vectors of field elements.
 - So computing the commitment $\langle \mathbf{x}, \mathbf{g} \rangle$ is a matrix-vector product.
 - Can be even faster than computing Pedersen commitments.
 - 0s are free to commit: any $x_i = 0$ can just be skipped, doesn't alter the commitment.

Ajtai commitments

- Recall Pedersen commitments:
 - Commitment key: n **random** group elements $(g_1, \dots, g_n)$.
 - The Pedersen commitment to a vector $\mathbf{x} = (x_1, \dots, x_n)$ of field elements (in additive group notation) is: $\langle \mathbf{x}, \mathbf{g} \rangle = \sum_{i=1}^n x_i \cdot g_i$.
- Ajtai commitments are similar, except that random group elements $(g_1, \dots, g_n)$ get replaced with vectors of random **ring** elements.
 - The ring elements are themselves vectors of field elements.
 - So computing the commitment $\langle \mathbf{x}, \mathbf{g} \rangle$ is a matrix-vector product.
 - Can be even faster than computing Pedersen commitments.
 - 0s are free to commit: any $x_i = 0$ can just be skipped, doesn't alter the commitment.
- Ajtai commitments are binding assuming Module SIS.
- Unlike elliptic curves they are plausibly post-quantum with no need to work over 256-bit fields.
- But a **major open direction** is to build better evaluation proofs for Ajtai commitments.

Good evaluation proofs for Ajtai commitments?

- LaBRADOR adapts Bulletproof-like techniques to the lattice setting.
 - ~50 KB proofs (asymptotically logarithmic) but slow verifier.
- Greyhound combines LaBRADOR with Hyrax-like techniques to reduce $\mathcal{V}$ time from linear to square-root.
- Neo gives a folding scheme for Ajtai commitments.
 - These can let $\mathcal{P}$ Ajtai-commit to many polynomials and provide a single evaluation proof for all of them.
- Neo has “pay per bit” commitment costs but LaBRADOR and Greyhound do not.
 - Due to using an embedding of field elements into ring elements, where “small” field elements do not map to “small” ring elements.
- Still open whether any of the above give a practical lattice-based commitment scheme with all the nice properties of, say, Dory.

Polynomial Commitments: Summary

Summary of Polynomial Commitments

- We have curve-based polynomial commitments like Dory with the below properties:
 1. 0s are **free** to commit to.
 2. Small values are **fast** to commit to.
 3. Polynomial evaluation proofs are fast to compute.
 - Roughly, **P** computes the correct evaluation, plus does square-root-sized multi-pairing.
 - The multi-pairing is a low-order cost for **P** (for large enough polynomials).
 4. **V** costs are logarithmic.
 5. Transparent.
 6. Square-root sized commitment key.

Summary of Polynomial Commitments

- We have curve-based polynomial commitments like Dory with the below properties:
 1. 0s are **free** to commit to.
 2. Small values are **fast** to commit to.
 3. Polynomial evaluation proofs are fast to compute.
 - Roughly, **P** computes the correct evaluation, plus does square-root-sized multi-pairing.
 - The multi-pairing is a low-order cost for **P** (for large enough polynomials).
 4. **V** costs are logarithmic.
 5. Transparent.
 6. Square-root sized commitment key.
- Lattice-based commitments promise to match or exceed the above.
 - Plus be post-quantum.
 - Plus allow for working over nicer fields.
 - ...but still much work to do (**major open direction**).
- Hashing-based commitments **do not** achieve free 0s.

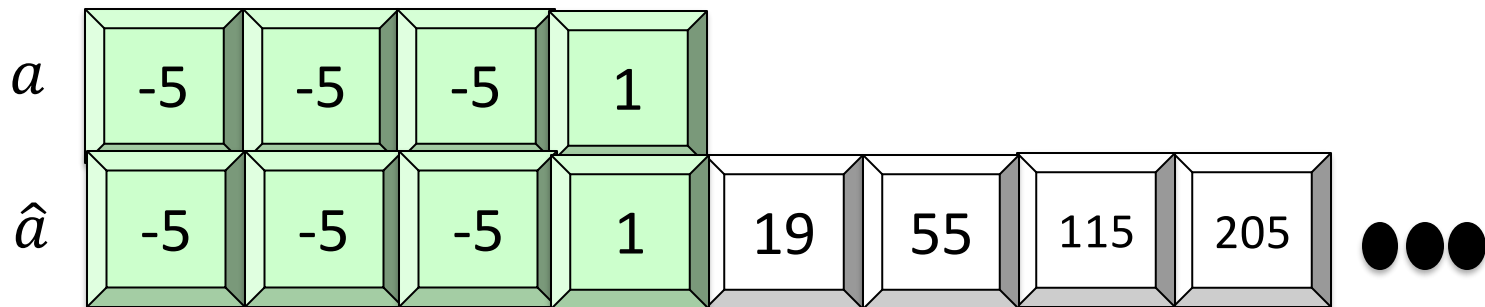
PIOPs using univariate polynomials and quotienting

Univariate Low-Degree Extension

- Let a be a length- n vector over a field F .
- Its low-degree extension $\hat{a}$ is the unique polynomial of degree less than n that matches a at n specified points.
- These points form an "interpolating set" $H \subseteq F$ of size n .
- Common choices for H :
 - $\{0, 1, \dots, n-1\}$
 - The n 'th roots of unity (i.e., roots of $x^n - 1$).

Univariate Low-Degree Extension

- Example ($n = 4$): Let $a = [-5, -5, -5, 1]$, $H = \{0, 1, 2, 3\}$.



$$\hat{a}(x) = x^3 - 3x^2 + 2x - 5$$

The Core Constraint: $a_i \cdot b_i = c_i$

- Let a , b , and c be length- n vectors committed via their low-degree extensions $\hat{a}$, $\hat{b}$, $\hat{c}$ using a univariate polynomial commitment scheme.
- **P** claims: For all $i \in H$,

$$\hat{a}(i) \cdot \hat{b}(i) - \hat{c}(i) = 0.$$

- Why this matters:
 - In SNARKs, a , b , and c often encode the left input, right input, and output of each multiplication gate in a circuit.
 - Verifying that all multiplication gates are correctly computed reduces to checking

$$\hat{a}(i) \cdot \hat{b}(i) = \hat{c}(i) \text{ for all } i \in H.$$

Quotienting via the Vanishing Polynomial

- Let $Z_H(X)$ be the **vanishing polynomial** for the set $H \subseteq F$.
 - The unique degree- n polynomial such that $Z_H(i) = 0$ for all $i \in H$.
 - Example: If H is the set of n' th roots of unity, then:

$$Z_H(X) = X^n - 1.$$

- Define: $g(X) = \hat{a}(X) \cdot \hat{b}(X) - \hat{c}(X)$.
- **KEY FACT:** $g(i) = 0$ for all $i \in H \Leftrightarrow Z_H(X)$ divides $g(X)$.
 - i.e., $g(X) = q(X) \cdot Z_H(X)$ for some polynomial $q(X)$.
- Degree bounds:
 - $\deg(g) \leq \deg(\hat{a}) + \deg(\hat{b}) \leq 2(n-1)$.
 - And $\deg(q) \leq \deg(g) - n \leq n-2$.

A Simple PIOP from This Divisibility Check

- The prover **commits to** the quotient polynomial $q(X)$
- V needs to check the **KEY FACT** identity: $g(X) = q(X) \cdot Z_H(X)$.
- To do this:
 - V samples random $r \in F$.
 - V checks whether $g(r) = q(r) \cdot Z_H(r)$, equivalently whether

$$\hat{a}(r) \cdot \hat{b}(r) - \hat{c}(r) = q(r) \cdot Z_H(r).$$

A Simple PIOP from This Divisibility Check

- The prover **commits to** the quotient polynomial $q(X)$
- V needs to check the **KEY FACT** identity: $g(X) = q(X) \cdot Z_H(X)$.
- To do this:
 - V samples random $r \in F$.
 - V checks whether $g(r) = q(r) \cdot Z_H(r)$, equivalently whether

$$\hat{a}(r) \cdot \hat{b}(r) - \hat{c}(r) = q(r) \cdot Z_H(r).$$

- The polynomial commitment scheme allows prover to open the committed polynomials $\hat{a}, \hat{b}, \hat{c}$ at r .
- V computes $Z_H(r)$ directly:
 - e.g., if $Z_H(X) = X^n - 1$ then $Z_H(r) = r^n - 1$, which V can compute in $O(\log n)$ time

Why This PIOP Is Slow for the Prover

- **P** must:
 - Compute the quotient polynomial $q(X)$, which requires FFTs and takes superlinear time.
 - Commit to $q(X)$.
- We have **almost no control** over q .
 - All we know about it is its degree is at most $n - 2$.
 - Even if $\hat{a}, \hat{b}, \hat{c}$ are **sparse** (mostly zero coefficients), q will generally be **dense** (all non-zero coefficients)
 - Even if $\hat{a}, \hat{b}, \hat{c}$ have small coefficients, q 's coefficients will not be.
- Takeaway:
 - Sparse polynomials (with small coefficients) are **fast to commit to**.
 - Quotient polynomials **don't have sparse or small coefficients**, so computing and committing to them is expensive.

The sum-check protocol

Why sum-check? Some intuition

- **Cryptography is expensive.**
 - Committing to lots of data is slow.
 - And the more committed data there is, the longer it takes to prove the data is “well-formed”.
- The sum-check protocol does not use cryptography **at all**.
 - It’s information-theoretically secure (until Fiat-Shamir is applied).
 - This makes it lightweight for **P**. The sum-check prover just adds and multiplies field elements.
- In SNARKs, it’s the perfect tool to let **P** commit to a minimal amount of data and then prove quickly that the committed data is well-formed.
- It’s a tool for offloading the hard work of checking the committed data to the prover, without requiring additional commitments, computation of dense/random quotient polynomials, etc.

More sum-check intuition

- Historical development: **Interactive Proofs** → **PCPs** → **SNARKs**.
 - IPs are interactive, PCPs are not.
 - Challenge in designing PCPs: “get rid of all interaction”, while keeping the PCP short.
 - IPs use multivariate polynomials and sum-check. PCPs use univariate polynomials and quotienting.
- The first SNARKs combined PCPs with commitment schemes.
 - **Kilian’s succinct interactive argument**: **P** Merkle-commits to a PCP, **V** sends PCP queries, **P** reveals only those PCP entries.
 - **Micali’s SNARK**: Apply Fiat-Shamir to get rid of the interaction.
- Today’s popular SNARKs are still descended from PCPs (univariate polynomials + quotienting).
- Why’d we take the interaction away to get a PCP, only to add it back in with Kilian’s transformation, and take it away a second time?
 - Interaction is a **resource** we should exploit for fast proving.
 - And just remove it once, with Fiat-Shamir.

Multilinear Extension Polynomials

Low-Degree and Multilinear Extensions

- Definition [**Extensions**]. Given a function $f: \{0,1\}^\ell \rightarrow \mathbb{F}$, a ℓ -variate polynomial g over $\mathbb{F}$ is said to **extend** f if $f(x) = g(x)$ for all $x \in \{0,1\}^\ell$.
- Definition [**Multilinear Extensions**]. Any function $f: \{0,1\}^\ell \rightarrow \mathbb{F}$ has a **unique** multilinear extension (MLE), denoted $\tilde{f}$.
 - Multilinear means the polynomial has degree at most 1 in each variable.
 - $(1 - x_1)(1 - x_2)$ is multilinear, $x_1^2 x_2$ is not.

$$f: \{0,1\}^2 \rightarrow \mathbb{F}$$

1	2
8	10

$$\tilde{f}: \mathbb{F}^2 \rightarrow \mathbb{F}$$

1	2	3	4	5	6
8	10	12	14	16	18
15	18	21	24	27	30
22	26	30	34	38	42
29	34	39	44	49	56

...

⋮

$$\tilde{f}(x_1, x_2) = (1 - x_1)(1 - x_2) + 2(1 - x_1)x_2 + 8x_1(1 - x_2) + 10x_1x_2$$

1	2	3	4	5	6
8	10	12	14	16	18
15	18	21	24	27	30
22	26	30	34	38	42
29	34	39	44	49	56

...

Can check:
 $\tilde{f}(0, 0) = 1$
 $\tilde{f}(0, 1) = 2$
 $\tilde{f}(1, 0) = 8$
 $\tilde{f}(1, 1) = 10$

⋮

Another (non-multilinear) extension of f : $g(x_1, x_2) = -x_1^2 + x_1x_2 + 8x_1 + x_2 + 1$

1	2	3	4	5	6
8	10	12	14	16	18
13	16	19	22	25	28
16	20	24	28	32	36
17	22	27	32	37	42

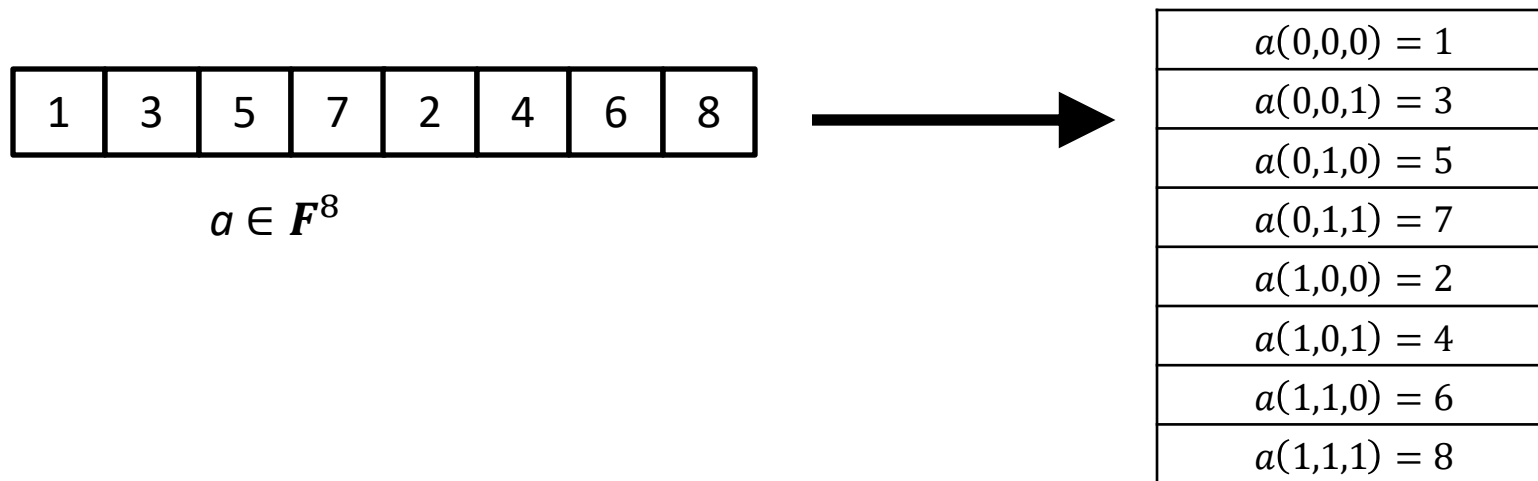
...

Can check:
 $g(0, 0) = 1$
 $g(0, 1) = 2$
 $g(1, 0) = 8$
 $g(1, 1) = 10$

⋮

Multilinear extensions of vectors

- We will interpret a vector a of length N as the evaluation table of a function over domain $\{0,1\}^{\log N}$, and refer to the multilinear of this function as the multilinear extension of a .



The sum-check protocol itself

Sum-Check Protocol [LFKN90]

- Input: V given oracle access to a ℓ -variate polynomial g over field $\mathbb{F}$.
- Goal: compute the quantity:

$$\sum_{b_1 \in \{0,1\}} \sum_{b_2 \in \{0,1\}} \dots \sum_{b_\ell \in \{0,1\}} g(b_1, \dots, b_\ell).$$

Sum-Check Protocol [LFKN90]

- **Start:** P sends claimed answer C_1 . The protocol must check that:

$$C_1 = \sum_{b_1 \in \{0,1\}} \sum_{b_2 \in \{0,1\}} \dots \sum_{b_\ell \in \{0,1\}} g(b_1, \dots, b_\ell).$$

Sum-Check Protocol [LFKN90]

- **Start:** P sends claimed answer C_1 . The protocol must check that:

$$C_1 = \sum_{b_1 \in \{0,1\}} \sum_{b_2 \in \{0,1\}} \dots \sum_{b_\ell \in \{0,1\}} g(b_1, \dots, b_\ell).$$

- **Round 1:** P sends **univariate** polynomial $s_1(X_1)$ claimed to equal:

$$H_1(X_1) := \sum_{b_2 \in \{0,1\}} \dots \sum_{b_\ell \in \{0,1\}} g(X_1, b_2, \dots, b_\ell)$$

Sum-Check Protocol [LFKN90]

- **Start:** **P** sends claimed answer C_1 . The protocol must check that:

$$C_1 = \sum_{b_1 \in \{0,1\}} \sum_{b_2 \in \{0,1\}} \dots \sum_{b_\ell \in \{0,1\}} g(b_1, \dots, b_\ell).$$

- **Round 1:** **P** sends **univariate** polynomial $s_1(X_1)$ claimed to equal:

$$H_1(X_1) := \sum_{b_2 \in \{0,1\}} \dots \sum_{b_\ell \in \{0,1\}} g(X_1, b_2, \dots, b_\ell)$$

- **V** checks that $C_1 = s_1(0) + s_1(1)$.

Sum-Check Protocol [LFKN90]

- **Start:** **P** sends claimed answer C_1 . The protocol must check that:

$$C_1 = \sum_{b_1 \in \{0,1\}} \sum_{b_2 \in \{0,1\}} \dots \sum_{b_\ell \in \{0,1\}} g(b_1, \dots, b_\ell).$$

- **Round 1:** **P** sends **univariate** polynomial $s_1(X_1)$ claimed to equal:

$$H_1(X_1) := \sum_{b_2 \in \{0,1\}} \dots \sum_{b_\ell \in \{0,1\}} g(X_1, b_2, \dots, b_\ell)$$

- **V** checks that $C_1 = s_1(0) + s_1(1)$.
- If this check passes, it is safe for **V** to believe that C_1 is the correct answer, so long as **V** believes that $s_1 = H_1$.
- How to check this? Just check that s_1 and H_1 agree at a random point r_1 .

Sum-Check Protocol [LFKN90]

- **Start:** **P** sends claimed answer C_1 . The protocol must check that:

$$C_1 = \sum_{b_1 \in \{0,1\}} \sum_{b_2 \in \{0,1\}} \dots \sum_{b_\ell \in \{0,1\}} g(b_1, \dots, b_\ell).$$

- **Round 1:** **P** sends **univariate** polynomial $s_1(X_1)$ claimed to equal:

$$H_1(X_1) := \sum_{b_2 \in \{0,1\}} \dots \sum_{b_\ell \in \{0,1\}} g(X_1, b_2, \dots, b_\ell)$$

- **V** checks that $C_1 = s_1(0) + s_1(1)$.
- If this check passes, it is safe for **V** to believe that C_1 is the correct answer, so long as **V** believes that $s_1 = H_1$.
- How to check this? Just check that s_1 and H_1 agree at a random point r_1 .
- **V** can compute $s_1(r_1)$ directly from **P**'s first message, but not $H_1(r_1)$.

Sum-Check Protocol [LFKN90]

- **Start:** **P** sends claimed answer C_1 . The protocol must check that:

$$C_1 = \sum_{b_1 \in \{0,1\}} \sum_{b_2 \in \{0,1\}} \dots \sum_{b_\ell \in \{0,1\}} g(b_1, \dots, b_\ell).$$

- **Round 1:** **P** sends **univariate** polynomial $s_1(X_1)$ claimed to equal:

$$H_1(X_1) := \sum_{b_2 \in \{0,1\}} \dots \sum_{b_\ell \in \{0,1\}} g(X_1, b_2, \dots, b_\ell)$$

- **V** checks that $C_1 = s_1(0) + s_1(1)$.
- **V** picks r_1 at random from $\mathbb{F}$ and sends r_1 to **P**.
- **Round 2:** They recursively check that $s_1(r_1) = H_1(r_1)$.

Sum-Check Protocol [LFKN90]

- **Start:** **P** sends claimed answer C_1 . The protocol must check that:

$$C_1 = \sum_{b_1 \in \{0,1\}} \sum_{b_2 \in \{0,1\}} \dots \sum_{b_\ell \in \{0,1\}} g(b_1, \dots, b_\ell).$$

- **Round 1:** **P** sends **univariate** polynomial $s_1(X_1)$ claimed to equal:

$$H_1(X_1) := \sum_{b_2 \in \{0,1\}} \dots \sum_{b_\ell \in \{0,1\}} g(X_1, b_2, \dots, b_\ell)$$

- **V** checks that $C_1 = s_1(0) + s_1(1)$.
- **V** picks r_1 at random from $\mathbb{F}$ and sends r_1 to **P**.
- **Round 2:** They recursively check that $s_1(r_1) = H_1(r_1)$.
i.e., that $s_1(r_1) = \sum_{b_2 \in \{0,1\}} \dots \sum_{b_\ell \in \{0,1\}} g(r_1, b_2, \dots, b_\ell)$.

Sum-Check Protocol [LFKN90]

- **Round ℓ (Final round):** **P** sends univariate polynomial $s_\ell(X_\ell)$ claimed to equal

$$H_\ell := g(r_1, \dots, r_{\ell-1}, X_\ell).$$

- **V** checks that $s_{\ell-1}(r_{\ell-1}) = s_\ell(0) + s_\ell(1)$.
- **V** picks r_ℓ at random and needs to check that $s_\ell(r_\ell) = g(r_1, \dots, r_\ell)$.
 - No need for more rounds. **V** can perform this check with one oracle query.

Costs of the sum-check protocol

- Total communication is $O(d\ell)$ field elements.
 - **P** sends ℓ messages, each a univariate polynomial of degree at most d . **V** sends $\ell - 1$ messages, each consisting of one field element.

Costs of the sum-check protocol

- Total communication is $O(d\ell)$ field elements.
 - P sends ℓ messages, each a univariate polynomial of degree at most d . V sends $\ell - 1$ messages, each consisting of one field element.
- V 's runtime is:
 $O(d\ell + [\textit{time required to evaluate } g \textit{ at one point}])$.

Sum-check Proving

- Recall: we're using sum-check to compute $\sum_{b_1 \in \{0,1\}} \dots \sum_{b_\ell \in \{0,1\}} g(b_1, \dots, b_\ell)$.

- Round i : P sends degree- d polynomial $s_i(X_i)$ claimed to equal:

$$\sum_{b_{i+1} \in \{0,1\}} \dots \sum_{b_\ell \in \{0,1\}} g(r_1, \dots, r_{i-1}, X_i, b_{i+1}, \dots, b_\ell)$$

- Suffices for P to specify $s_i(0), s_i(1), s_i(2), \dots, s_i(d)$

- Thus: Enough for P to evaluate g at all points of the form**

$$(r_1, \dots, r_{i-1}, \{0,1,2, \dots d\}, b_{i+1}, \dots, b_{\log n}): b_{i+1}, \dots, b_\ell \in \{0,1\}^{\ell-i}$$

Sum-check Proving

- Recall: we're using sum-check to compute $\sum_{b_1 \in \{0,1\}} \dots \sum_{b_\ell \in \{0,1\}} g(b_1, \dots, b_\ell)$.

- Round i : P sends degree- d polynomial $s_i(X_i)$ claimed to equal:

$$\sum_{b_{i+1} \in \{0,1\}} \dots \sum_{b_\ell \in \{0,1\}} g(r_1, \dots, r_{i-1}, X_i, b_{i+1}, \dots, b_\ell)$$

- Suffices for P to specify $s_i(0), s_i(1), s_i(2), \dots, s_i(d)$

- **Thus: Enough for P to evaluate g at all points of the form**

$$(r_1, \dots, r_{i-1}, \{0,1,2, \dots d\}, b_{i+1}, \dots, b_{\log n}): b_{i+1}, \dots, b_\ell \in \{0,1\}^{\ell-i}$$

- Two prover algorithms given in [CTY11] when g is a product of d multilinear polynomials:

- **Dense algorithm.** Runs in linear time, $O(2^\ell)$. Exploits that the number of terms being summed halves in every round, so total time is $2^\ell + 2^{\ell-1} + 2^{\ell-2} + \dots = O(2^\ell)$.
- **Sparse algorithm.** If only $m \ll 2^\ell$ terms of the original sum of non-zero, the sparse algorithm runs in $O(m)$ time per round, so $O(m\ell)$ time in total.
- The dense algorithm requires linear space $O(2^\ell)$, while the sparse algorithm works with a “streaming prover” in tiny space $O(\ell)$.

Sum-check-based PIOP for multiplication constraints

Proving $a_i \cdot b_i = c_i$ with sum-check

- Let a , b , and c be length- n vectors committed via their multilinear extensions $\tilde{a}$, $\tilde{b}$, $\tilde{c}$ using a multilinear polynomial commitment scheme.
- **P** claims: For all $x \in \{0, 1\}^{\log n}$,

$$\tilde{a}(x) \cdot \tilde{b}(x) - \tilde{c}(x) = 0.$$

Proving $a_i \cdot b_i = c_i$ with sum-check

- Let a , b , and c be length- n vectors committed via their multilinear extensions $\tilde{a}$, $\tilde{b}$, $\tilde{c}$ using a multilinear polynomial commitment scheme.

- P** claims: For all $x \in \{0, 1\}^{\log n}$,

$$\tilde{a}(x) \cdot \tilde{b}(x) - \tilde{c}(x) = 0.$$

- Let $\widetilde{eq}(x, y) = \prod_{i=1}^{\log n} (x_i y_i + (1 - x_i)(1 - y_i))$.

- This is the multilinear extension of the equality function:

- $(x_i y_i + (1 - x_i)(1 - y_i))$ maps $(0,0)$ and $(1,1)$ to 1 and maps $(1,0)$ and $(0,1)$ to 0.

- Fact:** **V** picks $r \in \mathbf{F}^{\log n}$ at random. Then up to soundness error $\frac{\log(n)}{|\mathbf{F}|}$, **P**'s original claim is equivalent to $0 = \sum_{x \in \{0,1\}^{\log n}} \widetilde{eq}(r, x) (\tilde{a}(x) \cdot \tilde{b}(x) - \tilde{c}(x))$.

Proving $a_i \cdot b_i = c_i$ with sum-check

- Let a , b , and c be length- n vectors committed via their multilinear extensions $\tilde{a}$, $\tilde{b}$, $\tilde{c}$ using a multilinear polynomial commitment scheme.

- P** claims: For all $x \in \{0, 1\}^{\log n}$,

$$\tilde{a}(x) \cdot \tilde{b}(x) - \tilde{c}(x) = 0.$$

- Let $\widetilde{eq}(x, y) = \prod_{i=1}^{\log n} (x_i y_i + (1 - x_i)(1 - y_i))$.

- This is the multilinear extension of the equality function:

- $(x_i y_i + (1 - x_i)(1 - y_i))$ maps $(0,0)$ and $(1,1)$ to 1 and maps $(1,0)$ and $(0,1)$ to 0.

- Fact:** **V** picks $r \in \mathbf{F}^{\log n}$ at random. Then up to soundness error $\frac{\log(n)}{|\mathbf{F}|}$, **P**'s original claim is equivalent

to $0 = \sum_{x \in \{0,1\}^{\log n}} \widetilde{eq}(r, y) (\tilde{a}(x) \cdot \tilde{b}(x) - \tilde{c}(x)).$

- Just apply the sum-check protocol to compute the boxed sum.

- Proof size is $O(\log n)$ and **P** needs just **$6n$ field multiplications (using the dense prover algorithm)**

- To perform its final check **V** needs to evaluate $\widetilde{eq}(r, r')$, $\tilde{a}(r')$, $\tilde{b}(r')$, $\tilde{c}(r')$.

- This is the core PIOP in Spartan.

- And, more or less, also in earlier works [Clover 2014] and [MIP=NEXP, BFL 1991]

Additional benefits of sum-check

- $\tilde{a}, \tilde{b}, \tilde{c}$ are typically **not** themselves committed.
 - They are **virtual polynomials**.
 - This means they are defined in terms of another polynomial $\tilde{z}$ that **is** committed.
- In “rank one constraint systems” (R1CS), $a = A \cdot z, b = B \cdot z, c = C \cdot z$, where $z \in \mathbf{F}^n$ is a committed “solution vector” and A, B, C are public “constraint matrices”.
 - Then $\tilde{a}(r') = \sum_{x \in \{0,1\}^{\log n}} \tilde{A}(r', x) \cdot \tilde{z}(x)$.

Additional benefits of sum-check

- $\tilde{a}, \tilde{b}, \tilde{c}$ are typically **not** themselves committed.
 - They are **virtual polynomials**.
 - This means they are defined in terms of another polynomial $\tilde{z}$ that **is** committed.
- In “rank one constraint systems” (R1CS), $a = A \cdot z, b = B \cdot z, c = C \cdot z$, where $z \in \mathbf{F}^n$ is a committed “solution vector” and A, B, C are public “constraint matrices”.
 - Then $\tilde{a}(r') = \sum_{x \in \{0,1\}^{\log n}} \tilde{A}(r', x) \cdot \tilde{z}(x)$
 - At the end of the previous slide when V needs to know $\tilde{a}(r')$, V can just apply the sum-check protocol again, to compute the boxed sum.
 - At the end of **that** sum-check, V needs to know $\tilde{A}(r', r'')$ and $\tilde{z}(r'')$.
 - $\tilde{z}(r'')$ can be obtained from the commitment to $\tilde{z}$.
 - $\tilde{A}(r', r'')$ can often be computed quickly by V on her own.

Leveraging Repeated Structure

- Let $a = A \cdot z$, where A is *block-diagonal*, i.e.,

$$A = \underbrace{\begin{bmatrix} S & 0 & \cdots & 0 \\ 0 & S & \ddots & \vdots \\ \vdots & \ddots & \ddots & 0 \\ 0 & \cdots & 0 & S \end{bmatrix}}_{T \text{ identical } 64 \times 64 \text{ blocks}} = \text{diag}(\underbrace{S, S, \dots, S}_{T \text{ times}}).$$

Here, $S \in \mathbb{F}^{64 \times 64}$ is a fixed 64×64 constraint matrix.

- Let $r' = (r'_{\text{blk}}, r'_{\text{copy}}) \in \{0, 1\}^{\log(64)} \times \{0, 1\}^{\log T}$.
- Then the multilinear extension of a satisfies

$$\tilde{a}(r') = \sum_{x \in \{0, 1\}^{\log(64)}} \tilde{S}(r'_{\text{blk}}, x_{\text{blk}}) \tilde{z}(x_{\text{blk}}, r'_{\text{copy}}).$$

- This sum is only over 64 terms, independent of the number of copies T of the base constraint system. We have essentially eliminated the cost of reducing $\tilde{a}(r')$ to an evaluation of $\tilde{z}$.

End Part 1

Key takeaways from Part 1

- 0s are free to commit to (with curve/lattice commitments).
- Small values are cheap to commit to (with appropriate curve/lattice/hashing commitments)
- Repeated structure can be exploited for performance gains.
- Part 2 of the talk:
 - Better leverage the above, especially when building zkVMs.
 - Where massively-repeated structure is intrinsic.

Part 2: zkVMs, batch-evaluation arguments, and streaming provers

zkVMs

- zkVMs are SNARKs that enable a prover to prove it correctly ran a computer program on a witness.
- The program is specified as **bytecode** compliant with some instruction set architecture (ISA).
 - Basically, machine code for a simple CPU (also called a Virtual Machine, or VM).
- This lets developers write computer programs in any high-level language that compiles to bytecode for the ISA.
- zkVM provers prove that they did two things correctly, over and over and over.
 - “Fetch-decode-execute”
 - Reads/writes to registers and RAM were answered faithfully.
- Key open challenges: **prover performance**, and **bugs**.

Current zkVM trends

1. A focus on the RISC-V instruction set architecture.
 - Why?
 - It's quite SNARK-friendly.
 - Simplest ISA designed by the computer architecture community (**without** SNARKs in mind).
 - Can re-use existing tooling and piggyback on other communities' efforts.
2. A focus on “real-time proving” for blockchains like Ethereum.
 - Today, processing one Ethereum block takes about 2^{30} RISC-V cycles.
 - Ethereum wants to increase the amount of work per block by 100x in the coming years.

mainnet EVM snarkification

zkVM	ISA	team	open source	mainnet EVM proofs	Ethproofs
Boojum 2.0	RISC-V	MatterLabs	ETA: June	✓	ETA: June
Ceno	RISC-V	Scroll	✓	ETA: Q3 (no recursion)	ETA: H2
Euclid (OpenVM)	RISC-V	Scroll	✓	✓	ETA: May
Ix	Lean 4	Argument	ETA: May	ETA: Q4 (no recursion)	ETA: H2
Jolt	RISC-V	a16z	✓	ETA: Q3 (no streaming)	ETA: H2
Keth (Cairo)	Cairo ISA	Kakarot	✓	ETA: Q2 (no continuations)	ETA: May
Ligetrn	WASM	Ligero	ETA: July	ETA: Q3 (no recursion)	ETA: H2
Linea EVM	EVM	Linea	✓	ETA: Q4 (no MPT)	ETA: H2
Miden VM	Miden ISA	Miden	✓	ETA: Q4 (no continuations)	ETA: H2
Nexus zkVM 3.0	RISC-V	Nexus	✓	ETA: Q3 (no recursion)	ETA: H2
Nock VM	Nock ISA	Zorp	✓	ETA: 2026	ETA: 2026
[redacted]	RISC-V	[redacted]	ETA: Q3	ETA: Q4 (???)	ETA: H2
o1VM	RISC-V	O(1) Labs	✓	ETA: Q3 (no recursion)	ETA: H2
OpenVM	RISC-V	Axiom	✓	✓	ETA: June
Petra	Petra ISA	Irreducible	✓	ETA: Q3 (no recursion)	ETA: H2
Pico	RISC-V	Brevis	✓	✓	ETA: June
powdrVM	RISC-V	powdr	✓	ETA: Q3 (no recursion)	ETA: H2
R0VM	RISC-V	RISC Zero	✓	✓	ETA: May
SP1	RISC-V	Succinct	✓	✓	✓
StarkV	RISC-V	StarkWare	ETA: Q4	ETA: Q4 (not started)	ETA: H2
Valida	Valida ISA	Lita	ETA: Q3	ETA: Q3 (no recursion)	ETA: H2
zkEngine	WASM	ICME	✓	✓	ETA: June
ZisK	RISC-V	Hermez	✓	✓	ETA: June
zkMIPS	MIPS	ZKM	✓	✓	✓
zkWASM	WASM	Delphinus	✓	✓	ETA: June

Source:
ethproofs.org
X account

Today's zkVM Zoo in a nutshell

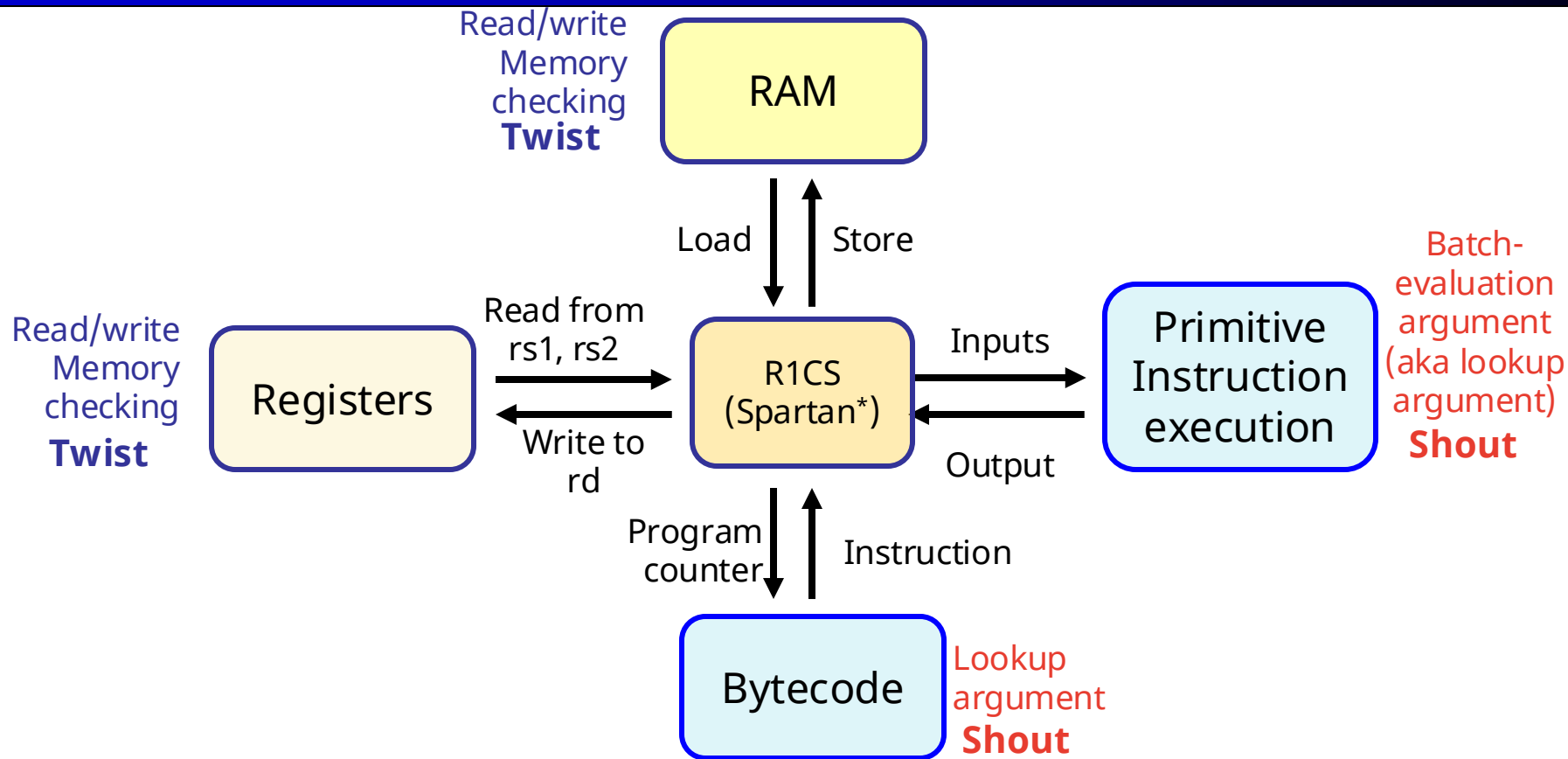
- The following zkVMs (some very early in development) have the sum-check protocol and multilinear polynomials at their core:
 - Jolt
 - Petra
 - RISC Zero v3
 - Ceno
- The rest are based on univariate polynomials.
 - Though some are exploring a “hybrid multi-univariate” approach
 - i.e., incorporating sum-check only partially.

The Jolt zkVM

What is Jolt (Just One Lookup Table)?

- RISC-V zkVM uniquely designed around the **sum-check protocol** and **lookup arguments** (aka **batch-evaluation arguments** aka **read-only memory-checking arguments**)
 - Primitive instruction execution handled by a sum-check-based batch-evaluation argument called Shout
 - The VM's registers and RAM are handled by a companion read/write memory checking argument called Twist
 - Minimal R1CS instance (~40 constraints per RISC-V cycle) ensures that at each cycle:
 - primitive instructions are evaluated on the correct input values
 - the correct values are written to and from registers and RAM
 - The program counter gets updated correctly
- **Fast** and relatively **simple**
 - state-of-the-art performance on CPU (GPU in progress)
 - Minimizes the amount of low-level circuit code written
 - Easier to audit and formally verify

Schematic of Jolt



Batch-evaluation arguments

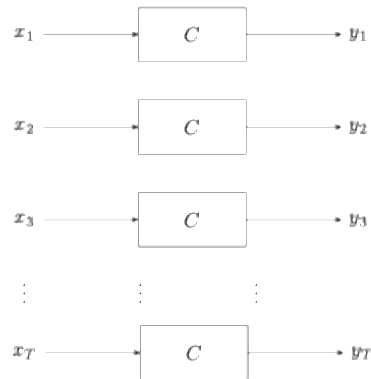
aka lookup arguments
aka memory-checking for read-only memories

Batch-evaluation arguments

- Let f be a primitive RISC-V instruction. f take two inputs (x, y) , each a B -bit value, and produces a B -bit output value z . In practice $B=32$.
 - i.e., x, y , and z are numbers in $\{0, 1, \dots, 2^B - 1\}$.
- Say P has committed to T input/output tuples, $(x_1, y_1, z_1), \dots, (x_T, y_T, z_T)$.
- P claims that for $y_i = f(x_i)$ for all $i = 1, \dots, T$.

Batch-evaluation arguments

- Let f be a primitive RISC-V instruction. f take two inputs (x, y) , each a B -bit value, and produces a B -bit output value z . In practice $B=32$.
 - i.e., x, y , and z are numbers in $\{0, 1, \dots, 2^B - 1\}$.
- Say $\mathbf{P}$ has committed to T input/output tuples, $(x_1, y_1, z_1), \dots, (x_T, y_T, z_T)$.
- $\mathbf{P}$ claims that for $y_i = f(x_i)$ for all $i = 1, \dots, T$.
- The “standard” way of doing this:
 - Let C be a circuit computing f .
 - Then consider the “data parallel circuit” that computes T copies of f .
 - Just apply any SNARK to this data parallel circuit.



Batch-evaluation arguments

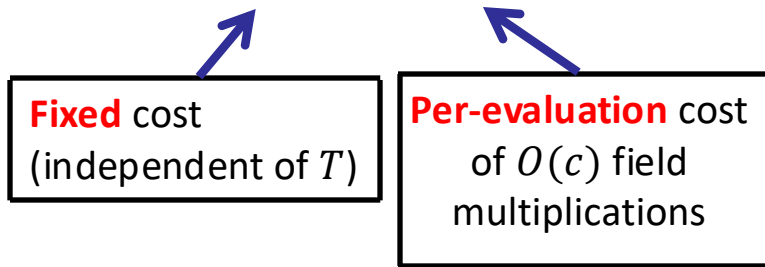
- Let f be a primitive RISC-V instruction. f take two inputs (x, y) , each a B -bit value, and produces a B -bit output value z . In practice $B=32$.
 - i.e., x, y , and z are numbers in $\{0, 1, \dots, 2^B - 1\}$.
- Say P has committed to T input/output tuples, $(x_1, y_1, z_1), \dots, (x_T, y_T, z_T)$.
- P claims that for $y_i = f(x_i)$ for all $i = 1, \dots, T$.
- The “standard” way of doing this:
 - Let C be a circuit computing f .
 - Then consider the “data parallel circuit” that computes T copies of f .
 - Just apply any SNARK to this data parallel circuit.
- This costs P at least 6 field multiplications **per gate**.
 - This is the fastest known implementation of sum-check-based SNARK provers for deterministic circuits.
 - If the circuit isn’t deterministic (i.e., it uses a lot “untrusted advice”), the cost will be even higher, as P needs to commit to the advice.

Batch-evaluation arguments

- Recall: P claims that for $z_i = f(x_i, y_i)$ for all $i = 1, \dots, T$.
- If the batch size T is big enough you can get rid of circuits entirely.
 - Batch-evaluation arguments are **faster and simpler** than SNARKs for circuit evaluation.

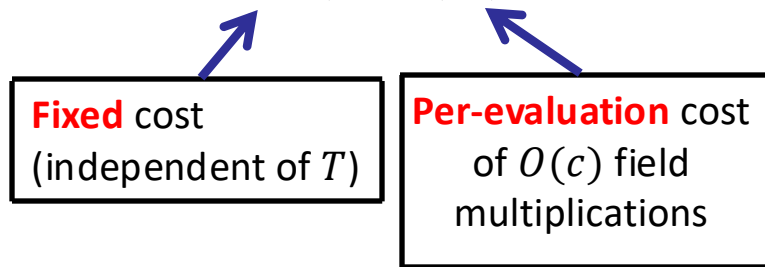
Batch-evaluation arguments

- Recall: P claims that for $z_i = f(x_i, y_i)$ for all $i = 1, \dots, T$.
- If the batch size T is big enough you can get rid of circuits entirely.
 - Batch-evaluation arguments are **faster and simpler** than SNARKs for circuit evaluation.
- Let $N = 2^{2B}$ denote the number of **possible** inputs to f .
 - P pays a "fixed cost" that depends on N and is amortized across all evaluations of f .
 - P pays a very low "**per-evaluation cost**".
 - E.g., P cost in Twist is $O(cN^{1/c}) + O(cT)$ field operations.



Batch-evaluation arguments

- Recall: **P** claims that for $z_i = f(x_i, y_i)$ for all $i = 1, \dots, T$.
- If the batch size T is big enough you can get rid of circuits entirely.
 - Batch-evaluation arguments are **faster and simpler** than SNARKs for circuit evaluation.
- Let $N = 2^{2B}$ denote the number of **possible** inputs to f .
 - P** pays a “fixed cost” that depends on N and is amortized across all evaluations of f .
 - P** pays a very low “**per-evaluation cost**”.
 - E.g., **P** cost in Twist is $O(cN^{1/c}) + O(cT)$ field operations for any desired $c > 0$.*



*For all RISC-V primitive instructions f , or more generally any f with “prefix-suffix structure”.

Batch-evaluation arguments

- Recall: **P** claims that for $z_i = f(x_i, y_i)$ for all $i = 1, \dots, T$.
- If the batch size T is big enough you can get rid of circuits entirely.
 - Batch-evaluation arguments are **faster and simpler** than SNARKs for circuit evaluation.
- Let $N = 2^{2B}$ denote the number of **possible** inputs to f .
 - **P** pays a "fixed cost" that depends on N and is amortized across all evaluations of f .
 - **P** pays a very low "**per-evaluation cost**".
 - E.g., **P** cost in Twist is $O(cN^{1/c}) + O(cT)$ field operations for any desired $c > 0$.
 - E.g., $N = 2^{64}$, $T = 2^{30}$, set $c = 4$, then fixed cost is only a few million field operations, negligible compared to the billion evaluations of f .
 - We'll see a lookup argument where the per-evaluation cost for $c = 4$ is well under 100 field multiplications.
 - This is faster than any circuit-based approach (unless f can be computed by a deterministic circuit with fewer than 15 gates, which is almost nothing).

The “old” way of designing batch-evaluation arguments: permutation-checking

Batch-evaluation via permutation-checking

- Recall: **P** claims that for $z_i = f(x_i, y_i)$ for all $i = 1, \dots, T$.
 - Each x_i, y_i, z_i are register values (i.e., one field element each).
 - N is the number of possible inputs to f , $c > 0$ is a parameter.
- Idea: identify two derived vectors **a** and **b**, both of length $\ell = O(cN^{1/c} + cT)$, such that **P**'s original claim is valid if and only if **a** and **b** are permutations of each other.
- Checking that **a** and **b** are permutations of each other:
 - **V** picks an $r \in \mathbf{F}$ at random and confirms that
$$\prod_{i=1}^{\ell} (a_i - r) = \prod_{i=1}^{\ell} (b_i - r).$$
- A SNARK for proving the value of a gigantic product like $\prod_{i=1}^{\ell} (a_i - r)$ is called a **grand product argument**.

Batch-evaluation via permutation-checking

- Recall: **P** claims that for $z_i = f(x_i, y_i)$ for all $i = 1, \dots, T$.
 - Each x_i, y_i, z_i are register values (i.e., one field element each).
 - N is the number of possible inputs to f , $c > 0$ is a parameter.
- Idea: identify two derived vectors **a** and **b**, both of length $\ell = O(cN^{1/c} + cT)$, such that **P**'s original claim is valid if and only if **a** and **b** are permutations of each other.
- Checking that **a** and **b** are permutations of each other:
 - **V** picks an $r \in \mathbf{F}$ at random and confirms that
$$\prod_{i=1}^{\ell} (a_i - r) = \prod_{i=1}^{\ell} (b_i - r).$$
- A SNARK for proving the value of a gigantic product like $\prod_{i=1}^{\ell} (a_i - r)$ is called a **grand product argument**.
- Two main options for sum-check-based grand product arguments:
 - [GKR 2008] protocol, as refined by [Thaler 2013].
 - Good: 3 field multiplications for **P** per term in the product.
 - Bad: $(\log \ell)^2$ proof size.
 - Spartan:
 - Good: $(\log \ell)$ proof size.
 - Bad: **P** commits to partial products, which are random field elements.

The Shout batch-evaluation argument

Shout batch-evaluation argument

- Recall: **P** claims that for $z_i = f(x_i, y_i)$ for all $i = 1, \dots, T$.
- In Shout, the vectors z , x , and y are **not committed**, they are **all virtual**.
- What is committed: each (x_i, y_i) in **one-hot format**.
 - The one-hot encoding of (x_i, y_i) is a length- 2^{2B} vector with exactly one entry equal to 1 and all the rest equal to 0.
 - Formally, let $\text{ra}(k, i): \{0,1\}^{2B} \times \{0,1\}^{\log T} \rightarrow \{0,1\}$ and suppose that for any fixed $i \in \{0,1\}^{\log T}$, $\text{ra}(k, i) = 1$ for **exactly one** $k \in \{0,1\}^{2B}$.
 - $\text{ra}(\cdot, i)$ is a length- 2^{2B} vector called the **one-hot-encoding of** (x_i, y_i) .
- $\tilde{\text{ra}}$ is **huge but sparse**.
- Why is committing to $\tilde{\text{ra}}$ is useful?
 - It ensures the constraint $z_i = f(x_i, y_i)$ becomes degree a degree-2 expression amenable to sum-check:
 - $\tilde{z}(i) = \sum_{k \in \{0,1\}^{2B}} \tilde{\text{ra}}(k, i) \cdot \tilde{f}(k)$.

Shout batch-evaluation argument

- Recall: $\tilde{z}(i) = \sum_{k \in \{0,1\}^{64}} \tilde{r}a(k, i) \cdot \tilde{f}(k)$.
- We have made $\tilde{z}$ virtual!
- When V wants to evaluate $\tilde{z}(r)$ it just applies sum-check to compute
$$\sum_{k \in \{0,1\}^{64}} \tilde{r}a(k, r) \cdot \tilde{f}(k).$$
- At the end of sum-check, V has to evaluate $\tilde{r}a(r', r)$ and $\tilde{f}(r')$ for some random r' .
- V gets $\tilde{r}a(r', r)$ from the commitment to $\tilde{r}a$, and $\tilde{f}$ can be computed in about 64 field operations for all RISC-V primitive instructions
- Remaining issues for P :
 1. The sum is over 2^{64} terms. How do we apply sum-check to such a big sum?
 2. $\tilde{r}a$ has size $2^{64} \cdot T$. Isn't this way too big to commit to?

Shout batch-evaluation argument

- Recall: $\tilde{z}(i) = \sum_{k \in \{0,1\}^{64}} \tilde{r}a(k, i) \cdot \tilde{f}(k)$.
- We have made $\tilde{z}$ virtual!
- When V wants to evaluate $\tilde{z}(r)$ it just applies sum-check to compute
$$\sum_{k \in \{0,1\}^{64}} \tilde{r}a(k, r) \cdot \tilde{f}(k).$$
- At the end of sum-check, V has to evaluate $\tilde{r}a(r', r)$ and $\tilde{f}(r')$ for some random r' .
- V gets $\tilde{r}a(r', r)$ from the commitment to $\tilde{r}a$, and $\tilde{f}$ can be computed in about 64 field operations for all RISC-V primitive instructions
- Remaining issues for P :
 1. The sum is over 2^{64} terms. How do we apply sum-check to such a big sum?
 2. $\tilde{r}a$ has size $2^{64} \cdot T$. Isn't this way too big to commit to?
- For both issues, P is saved by the extreme, structured sparsity of $\tilde{r}a$.

Addressing the first issue

- Fact: for any $r \in \mathbf{F}^{\log T}$, $\tilde{r}a(k, r)$ is non-zero for only at most T values of $k \in \{0,1\}^{2B}$.
- So the **sparse sum-check proving algorithm** runs in time $O(TB)$.
- But if $\tilde{f}$ is “structured” in a way that all RISC-V primitive instructions are, we can bring this down to $O(T)$.

Addressing the second issue

- Recall $\tilde{r}a(k, i)$ has size $2^{64} \cdot T$, but each “column” $i \in \{0,1\}^{\log T}$ has exactly one 1.
- We can use this to make $\tilde{r}a$ virtual too!
- Instead, have $\mathbf{P}$ commit to $\tilde{r}a_1, \dots, \tilde{r}a_c: \{0,1\}^{64/c} \times \{0,1\}^{\log T} \rightarrow \{0,1\}$.
- Then vectors are each one-hot as well and only have size $2^{64/c} \cdot T$.
- When $c = 4$, these vectors, while still very large, are short enough to commit to with, say, Dory .
 - Need free commitment to 0s, sublinear-size commitment key, fast polynomial evaluation proofs to make this work.
- Then $\tilde{r}a(k_1, \dots, k_c, r) = \sum_{i \in \{0,1\}^{\log T}} \prod_{j=1}^c \tilde{r}a_j(k_j, i)$

Controlling prover memory usage

The prevailing way of controlling P memory costs

- Naively implemented, a zkVM “applied monolithically” to a $\$T\$$ -cycle VM execution uses space linear in $\$T\$$.
 - *Concretely, 2^{30} cycles leads to petabytes of space.*
- *The “old” way of controlling memory usage is called “continuations”*

Main Technical Takeaways for SNARK Design

Main Technical Takeaways

- The **sum-check protocol is key** to fast SNARK provers.
- But not all uses of sum-check are equal.
 - Fully leverage **repeated structure** for vast performance and simplicity benefits.
 - i.e., zkVMs shouldn't use circuits, but rather (sum-check-based) batch-evaluation arguments.
- The **interplay** between PCses and PIOPs is key.
 - A key challenge in SNARK design is “sparsity”.
 - E.g., if you do T lookups into a memory of size K , a priori there are $K \cdot T$ (memory cell, timestep) possibilities but only T actually occur.
 - Old approach to dealing with sparsity: reduce to a dense problem of size $O(K + T)$.
 - New approach: leave things sparse and don't pay for 0s.
 - **PIOP prover** must avoid “paying” for 0s and must avoid committing to ‘big’ values.
 - **Commitment scheme** must achieve pay-per-bit commitment costs.
- Major transitions are underway:
 - Univariate/quotienting → multilinear/sum-check; hashing-based commitments → curves and lattices; recursion for prover memory control → streaming prover.

Open Research Directions

Open Research Directions

- **Lattice-based polynomial commitments**
 - Goal: Develop something we can switch Jolt over to (from curve-based commitments).
 - Want plausible post-quantum security, work over (a 128-bit extension of) the Goldilocks field, with minimal extra costs and complications.
- **Formal Verification**
 - Today, bugs are **everywhere** in SNARKs.
 - These protocols are so complicated that the **only** way we can have confidence in their correctness is formal verification.
 - This is a huge challenge for “programmable cryptography” in general (MPC, FHE, etc).
 - SNARKs are just the canary in the coalmine.
- **Fiat-Shamir security**
 - We do not actually understand the plain-model security of SNARKs that invoke Fiat-Shamir transformation in the plain model.
 - FS is now known to be **insecure for natural SNARKs**, albeit for **contrived circuits**.
 - Today, bugs are the biggest security concern by far. But in a few years, formal verification efforts will have borne fruit. Then Fiat-Shamir security may be the dominant concern.

Thank you!

More Open Research Directions

- Streaming zkVM provers for T -cycle VM executions with $O(T)$ instead of $O(T \log T)$ field multiplications?
- Clarity on the dividing line between PIOPs and PCSes?
 - KZG evaluation proof is just the quotienting PIOP, where the verifier's random challenge is the toxic waste τ “hidden in the exponent” of the powers-of- τ trusted setup.
 - Bulletproofs evaluation proof is just the sum-check protocol, applied to compute the inner product of two vectors, where the entries of one vector are “hidden in the exponents” of the commitment key.
 - Similar connection for hashing-based commitments?